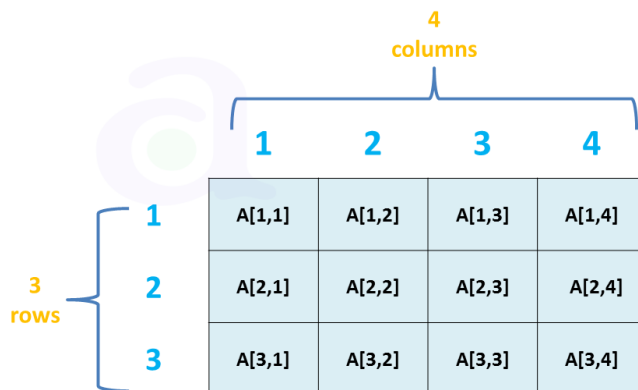


Two-dimension Array in Data Structure

A two-dimension $m \times n$ array, A is it collection of $m.n$ data elements such that each elements is specified by pair of integers (such as J,K) called subscripts, with the property that,

$$1 \leq J \leq m, 1 \leq K \leq n$$

There is a standard way of drawing a two-dimensional $m \times n$ array A where, the elements of A form a rectangular array with m rows and n columns and where, the elements $A[J.K]$ appears in row J and column K ,



The diagram illustrates a 3x4 array. On the left, a bracket labeled '3 rows' in orange groups the row indices 1, 2, and 3. Above the table, a bracket labeled '4 columns' in orange groups the column indices 1, 2, 3, and 4. The table itself has three rows and four columns, with each cell containing an element identifier like A[1,1].

	1	2	3	4
1	$A[1,1]$	$A[1,2]$	$A[1,3]$	$A[1,4]$
2	$A[2,1]$	$A[2,2]$	$A[2,3]$	$A[2,4]$
3	$A[3,1]$	$A[3,2]$	$A[3,3]$	$A[3,4]$

Fig: Two-dimension 3x4 array

Suppose, A is a two-dimensional $m \times n$ array. The first dimension of A contains the index set $1, \dots, m$ with lower bound 1 and upper bound m , and the second dimension of A contains the index set $1, 2, \dots, n$ with lower bound 1 and upper bound n . The length of a dimension is the number of integers in its index set. The pair of lengths **$m \times n$** is called **size of the array**.

Programming language allows one to define multi-dimensional array in which lower bounds are not 1. However, the index set for each dimension is still consists of consecutive integers from the

lower bound to the upper bound of the dimension. The length of a given dimension can be obtained from the formula,

$$\text{Length} = \text{UB} - \text{LB} + 1$$

For two-dimensional array we can find the length of the array by the following procedure,

1. First we will find the length of one-dimensional row array by the formula,
 $L_1 = \text{UB}_1 - \text{LB}_1 + 1$
 2. Do this again one-dimensional column array.
 $L_2 = \text{UB}_2 - \text{LB}_2 + 1$
 3. Length of array (number of elements in two-dimensional array)
 $L = L_1 \times L_2$
- Example

Given array `int A(2 : 5, - 3 : 1)`

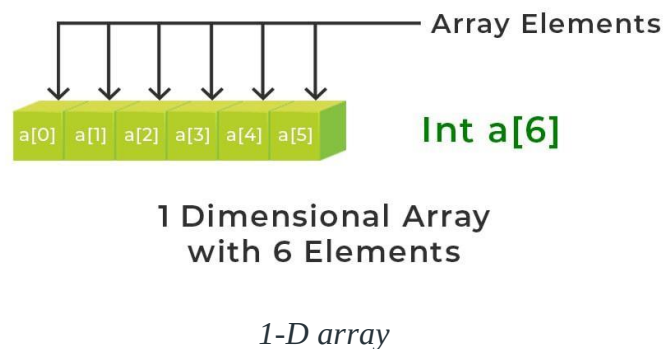
1. The length of row one-dimensional array ($5 - 2 + 1 = 4$)
2. The length of column one-dimensional array ($1 - (-3) + 1 = 5$)
3. Length of given array = $4 \times 5 = 20$

So, there are 20 elements on the given array.

Calculating the address of any element In the 1-D array:

A 1-dimensional array (or single-dimension array) is a type of [linear array](#). Accessing its elements involves a single subscript that can either represent a row or column index.

Example:



To find the address of an element in an array the following formula is used-

$$\text{Address of } A[I] = B + W * (I - LB)$$

I = Subset of element whose address to be found,

B = Base address,

W = Storage size of one element store in any array(in byte),

LB = Lower Limit/Lower Bound of subscript(If not specified assume zero).

Example: Given the base address of an array **A[1300 1900]** as **1020** and the size of each element is 2 bytes in the memory, find the address of **A[1700]**.

Solution:

Given:

Base address $B = 1020$

Lower Limit/Lower Bound of subscript $LB = 1300$

Storage size of one element store in any array $W = 2$ Byte

Subset of element whose address to be found $I = 1700$

Formula used:

$$\text{Address of } A[I] = B + W * (I - LB)$$

Solution:

$$\text{Address of } A[1700] = 1020 + 2 * (1700 - 1300)$$

$$= 1020 + 2 * (400)$$

$$= 1020 + 800$$

$$\text{Address of } A[1700] = 1820$$

Calculate the address of any element in the 2-D array:

The 2-dimensional array can be defined as an array of arrays. The 2-Dimensional arrays are organized as matrices which can be represented as the collection of rows and columns as array[M][N] where M is the number of rows and N is the number of columns.

Example:

Columns				
Rows	2D Array	0	1	2
	0	a[0][0]	a[0][1]	a[0][2]
	1	a[1][0]	a[1][1]	a[1][2]
	2	a[2][0]	a[2][1]	a[2][2]

2-D array

To find the address of any element in a **2-Dimensional** array there are the following two ways-

1. Row Major Order
2. Column Major Order

1. Row Major Order:

Row major ordering assigns successive elements, moving across the rows and then down the next row, to successive memory locations. In simple language, the elements of an array are stored in a Row-Wise fashion.

To find the address of the element using row-major order uses the following formula:

$$\text{Address of } A[I][J] = B + W * ((I - LR) * N + (J - LC))$$

I = Row Subset of an element whose address to be found,

J = Column Subset of an element whose address to be found,

B = Base address,

W = Storage size of one element store in an array(in byte),

LR = Lower Limit of row/start row index of the matrix(If not given assume it as zero),

LC = Lower Limit of column/start column index of the matrix(If not given assume it as zero),

N = Number of column given in the matrix.

Example: Given an array, **arr[1.....10][1.....15]** with base value **100** and the size of each element is **1 Byte** in memory. Find the address of **arr[8][6]** with the help of row-major order.

Solution:

Given:

Base address B = 100

Storage size of one element store in any array W = 1 Bytes

Row Subset of an element whose address to be found I = 8

Column Subset of an element whose address to be found J = 6

Lower Limit of row/start row index of matrix LR = 1

Lower Limit of column/start column index of matrix = 1

$$\begin{aligned}\text{Number of column given in the matrix } N &= \text{Upper Bound} - \text{Lower Bound} + 1 \\ &= 15 - 1 + 1 \\ &= 15\end{aligned}$$

Formula:

$$\text{Address of } A[I][J] = B + W * ((I - LR) * N + (J - LC))$$

Solution:

$$\begin{aligned}\text{Address of } A[8][6] &= 100 + 1 * ((8 - 1) * 15 + (6 - 1)) \\ &= 100 + 1 * ((7) * 15 + (5)) \\ &= 100 + 1 * (110)\end{aligned}$$

$$\text{Address of } A[I][J] = 210$$

2. Column Major Order:

If elements of an array are stored in a column-major fashion means moving across the column and then to the next column then it's in column-major order. To find the address of the element using column-major order use the following formula:

$$\text{Address of } A[I][J] = B + W * ((J - LC) * M + (I - LR))$$

I = Row Subset of an element whose address to be found,

J = Column Subset of an element whose address to be found,

B = Base address,

W = Storage size of one element store in any array(in byte),

LR = Lower Limit of row/start row index of matrix(If not given assume it as zero),

LC = Lower Limit of column/start column index of matrix(If not given assume it as zero),

M = Number of rows given in the matrix.

Example: Given an array **arr[1.....10][1.....15]** with a base value of **100** and the size of each element is **1 Byte** in memory find the address of **arr[8][6]** with the help of column-major order.

Solution:

Given:

Base address $B = 100$

Storage size of one element store in any array $W = 1$ Bytes

Row Subset of an element whose address to be found $I = 8$

Column Subset of an element whose address to be found $J = 6$

Lower Limit of row/start row index of matrix $LR = 1$

Lower Limit of column/start column index of matrix $= 1$

Number of Rows given in the matrix $M = \text{Upper Bound} - \text{Lower Bound} + 1$
 $= 10 - 1 + 1$
 $= 10$

Formula: used

Address of $A[I][J] = B + W * ((J - LC) * M + (I - LR))$

Address of $A[8][6] = 100 + 1 * ((6 - 1) * 10 + (8 - 1))$
 $= 100 + 1 * ((5) * 10 + (7))$
 $= 100 + 1 * (57)$

Address of $A[I][J] = 157$

From the above examples, it can be observed that for the same position two different address locations are obtained that's because in row-major order movement is done across the rows and then down to the next row, and in column-major order, first move down to the first column and then next column. So both the answers are right.

So it's all based on the position of the element whose address is to be found for some cases the same answers is also obtained with row-major order and column-major order and for some cases, different answers are obtained.